OBSERVACIONES DE LA PRÁCTICA

Jimmy Mújica Delgado - 202224528
Samuel Esteban Díaz Arias - 202522112

Preguntas de análisis

1. ¿Qué diferencia existe entre las alturas de los dos árboles (BST y RBT)?
   Con la misma cantidad de datos el árbol RBT queda más abajo que el BST. Ya que, el BST no se balancea, así que su altura depende en el orden de llegada de las llaves; por ejemplo, cuando llegan casi ordenadas, se alarga hacía un lado imitando una lista encadenada. El RBT se organiza con ambas de sus rotaciones y cambios de color cada vez que se hace una inserción, y su altura se mantiene cerca de log2(n) y con un máximo de 2*log2(n)
2. ¿Percibe alguna diferencia entre la ejecución de los dos árboles (RBT y BST)? ¿Por qué pasa esto?
   Sí. Las búsquedas y consultas que son por rango resultan mucho más rápidas en el RBT, ya que no necesitan recorrer tantos niveles. La carga puede tardar casi lo mismo, ya que cada inserción del RBT tiene que hacer un trabajo extra para mantener el balance, lo cual es un costo adicional que se compensa en su búsqueda.
3. ¿Existe alguna diferencia de complejidad entre los dos árboles (RBT y BST)? Justifique su respuesta.
   Si hay una diferencia entre las dos, pues en el BST, buscar, insertar y consultar rangos cuesta O(h). En el peor caso h = n, en el caso de que las llaves estén ordenadas, se vuelve una lista O(n). Por el otro lado, el RBT garantiza $h \leq 2*\log2(n)$, razón por la cual las operaciones serían O(logn) hasta en el peor caso.
4. ¿Existe alguna manera de cargar los datos en un árbol RBT de tal forma que su funcionamiento mejore? Si es así, mencione cuál.
   El orden de carga no cambia la complejidad, pues su altura no cambia, como se justificó en el primer punto. Sin embargo, si se puede reducir el costo de carga:
   - Cargando en orden aleatorio, ya que si es en orden crea excesivas rotaciones por el hecho de que caerían en el mismo lado
   - Agrupar antes de insertar, juntar los crímenes por llave en una tabla de hash para luego solo usar "put” por llave